How the Project's Technology Works Together

AI Debate and Interview Coach: A Beginner's Guide

This guide follows the data through the project: from your voice, to text, to the AI, and then to feedback.

The Main Flow

```
YOU SPEAK
   |
   v
Browser Speech Recognition
   |  converts audio into words (a transcript)
   v
Next.js + React user interface
   |  sends transcript or chat history over HTTP
   v
Python + FastAPI backend
   |-- Debate / interview: sends prompt and conversation to Groq API
   |                              |
   |                              v
   |                    Configured language model (LLM)
   |                              |
   |<--------------------- generated text reply
   |
   |-- Speech practice: counts words, fillers, and hedge phrases
   |        |-- optional Claude API scores transcript and suggests tips
   |        `-- otherwise local rules calculate approximate scores
   |
   |-- SQLite saves completed speech-practice results
   v
Next.js + React displays the reply, scores, and suggestions
   |
   `-- Browser Speech Synthesis can read the AI reply aloud
```

Who Does What?

Step	Technology	What it does here
1. Website screen	**Next.js 14 + React 18**	Displays Practice, Debate, Mock Interview, and Dashboard pages. React updates the screen as a session progresses.
2. Browser code and data shapes	**TypeScript 5**	Adds types for frontend data such as messages and scores. It helps catch some coding mistakes during development.
3. Website styling	**Tailwind CSS 3**	Applies layout, colors, spacing, and responsive styling to the React pages.

4. Speech to text (STT)	**Browser Web Speech API: SpeechRecognition**	Listens to the microphone and converts the student's speech into transcript text. This is a browser feature, not Groq and not a model implemented by the Python backend. Its support and audio handling depend on the browser/device.
5. Send text to the server	**HTTP + JSON; Next.js rewrite**	Sends transcript, topic, or chat history from the browser to a FastAPI route. The Next.js rewrite forwards `/api/...` requests to the local backend, normally on port 8000.
6. Receive and validate requests	**Python + FastAPI + Pydantic**	FastAPI routes the request to the correct operation. Pydantic checks the shape and types of request/response data.
7. Generate debate and interview replies	**Groq API + configured LLM**	The backend sends the topic or interview history and instructions to Groq. The configured LLM generates a text reply, which the backend returns to the website. The code's default model name is `openai/gpt-oss-120b`; `GROQ_MODEL` can override it.
8. Read replies aloud (text to speech / TTS)	**Browser Web Speech API: SpeechSynthesis**	Takes the AI's returned text and speaks it through the user's browser/device. This is separate from the LLM: the LLM writes the answer; SpeechSynthesis reads it aloud.
9. Evaluate practice transcript	**Python regex; optional spaCy**	Checks transcript text for configured filler words and hedging phrases, and counts basic word/sentence statistics. spaCy can help with text processing if installed; a simpler fallback is present.
10. Create feedback and scores	**Optional Anthropic Claude API OR local scoring rules**	If `ANTHROPIC_API_KEY` and the `anthropic` package are available, Claude evaluates the transcript and returns scores/tips. If not, fixed local formulas estimate the scores and create tips. The scoring endpoint does not analyze the original audio waveform.
11. Save practice history	**SQLite**	Stores completed speech-analysis sessions: scores, tips, transcript, and date. The dashboard reads these records. The sessions route does not save debate/interview chat history.
12. Show history charts	**Recharts**	Draws charts in the React dashboard from saved SQLite session data returned by the backend.
13. Optional webcam estimates	**face-api.js in the browser**	Runs face/landmark/expression estimates locally in the page while the camera is active. These estimates are separate from the backend transcript score and are not scientifically validated judgments of confidence.
14. Read uploaded resumes	**pypdf + Python standard library**	pypdf extracts text from PDFs. Built-in ZIP/XML tools extract text from DOCX. Plain TXT files are read as

		text. The extracted text is sent to the LLM to tailor interview questions.
15. Run the Python web server	**Uvicorn**	Starts and serves the FastAPI application locally, normally at `http://127.0.0.1:8000`.

Two Different AI Jobs

The project has two separate kinds of AI-related work. Keeping them separate helps explain the architecture:

Conversation generation

For **Debate** and **Mock Interview**, the FastAPI backend sends instructions and conversation context to the **Groq API**. Groq is the service provider. The configured LLM generates the next text reply. The frontend displays that reply, and browser SpeechSynthesis may read it aloud.

Speech-practice evaluation

For **Practice**, the browser's SpeechRecognition first turns microphone speech into text. The backend evaluates that text. It counts filler/hedge phrases with regex and may use optional spaCy for basic text statistics. It can ask **Anthropic Claude** for rubric-style scoring if configured; otherwise, the backend uses local formulas. The result includes five estimated scores and tips, then is saved to SQLite.

So, the project does **not** send raw voice to Groq for pronunciation or tone analysis. Groq is used for generating debate/interview text. The practice evaluator receives a transcript.

Important Names: Groq, Grok, and Claude

- **Groq** is the API provider called by the debate and interview backend routes.
- **Grok** is a separate xAI model family. This project does not call xAI Grok.
- **Claude** is Anthropic's model family. It is an optional provider for speech transcript scoring.
- **LLM** means "large language model." It describes a type of AI model, not a company.

The Anthropic scoring dependency is optional and is not listed in the current `backend/requirements.txt`. Without its Python package and API key, transcript scoring falls back to local rules. Confirm which scoring mode is active before a demo.

What the Scores Can and Cannot Tell You

The scorecard reports clarity, confidence, structure, fluency, and engagement. Since the backend is given text, its scores are estimates based on the words and rules/model feedback. It cannot directly hear vocal pitch, volume, pronunciation, real pauses, or speaking pace from the audio. A pause estimate in the current backend is derived from filler-word count. Treat the score as practice guidance, not an objective grade.

The face model estimates visible face/expression patterns. It cannot know a person's true feelings, honesty, or confidence. Lighting, camera position, and model limitations can affect results.

Short Explanation for a Professor

> *"The browser's Web Speech API converts the student's speech to text. The Next.js and React frontend sends this transcript or chat history to our Python FastAPI backend. For debates and interviews, the*

> *backend calls Groq, where a configured LLM generates a response. The browser can then read that response aloud using SpeechSynthesis. For speech practice, the backend checks transcript words using regular expressions and can optionally ask Anthropic Claude to score it; if Claude is not configured, local rules provide approximate scores and tips. SQLite saves speech-practice history, and Recharts displays it."*

Major Limitations to Mention

- Speech recognition depends on browser support and can mishear accents, technical terms, and noisy speech.
- Debate and interview replies need an internet connection and valid Groq configuration; an LLM can produce incorrect claims.
- Transcript scores are approximate and are not validated scientific measurements.
- Optional Claude scoring needs a separate package and API key; otherwise local heuristic scoring is used.
- The backend does not evaluate the raw voice for tone, pronunciation, or real silence duration.
- Webcam analysis is an estimate, not a reliable measure of internal emotion or attention.
- SQLite is local prototype storage; there is no account system or multi-user privacy separation.